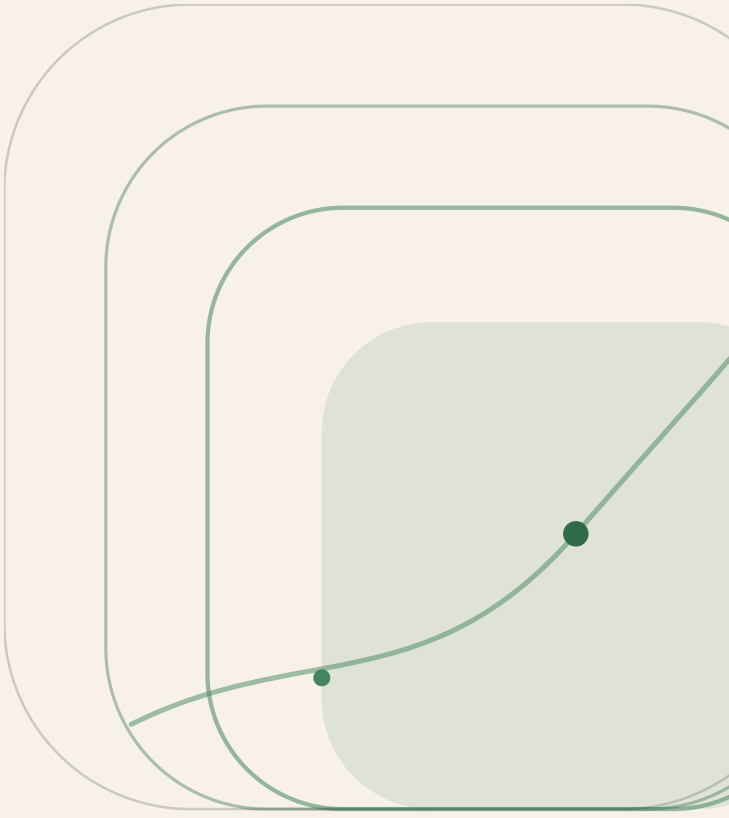
greecon
SH.P.K

GUIDE 01 · PART 2

Founder Go-Live Guide

Put the platform on the internet: Railway, the two addresses, the first sign-in, email and text messages, the demo, the outside checks and backups.

FOR	The founder, once. Nothing technical is assumed
PRINT	Once, before the first customer. The last page tears off as the go-live checklist
COVERS	Twelve steps of clicking and pasting, what you should see at each, what to do if you do not, and the checklist



Version 1.0 · 26 October 2026

Greecon Sh.p.k · Durana Tech Park · info@greecon.earth

Contents

Part 2 Platform go-live: the founder 3
What you need before you start 3
Step 1 Three addresses, decided 4
Step 2 Railway: the project and its database 4
Step 3 Railway: the API service 4
Step 4 Railway: the web service 6
Step 5 DNS: the two addresses 6
Step 6 First sign-in, and removing the starting password 6
Step 7 Email (Resend) 7
Step 8 Text messages (Twilio) 7
Step 9 Greecon's own morning check 8
Step 10 The demo 8
Step 11 Watching it 9
Step 12 The rehearsal, then the checklist 10
If something goes wrong 10
Changing things later 10
What not to do 11
Go-live checklist 12

PART 2

Platform go-live: the founder

For: the founder, once. **Takes:** three to four hours the first time, all of it clicking and pasting. **You need to know nothing technical.** Every step says what you should see when it worked, and what to do if you do not see it. Where a button has a different name today, look for the one that means the same.

When this is finished, a prospect can open the demo, a customer can sign in, a project manager can set a site up, a box at a hotel can report in, and a person's phone rings when a tank runs dry.

Read Launch decisions (document 27) first if you want to know why things are as they are.

What you need before you start

You need	Why	Where
A GitHub login that can see `greecon-tech/app`	The platform is built from it	github.com
A credit card for Railway	Hosts the platform and its database	railway.com
Access to where the `greecon.earth` addresses are managed (the registrar or DNS login)	Two web addresses and email proof need adding there	wherever you bought the domain
An email address that can receive mail from `@greecon.earth` (`info@greecon.earth`)	New requests for access, and replies to alert emails, go there	your mail
A password manager	Every password and key below goes in it, not in a note or a chat	1Password, Bitwarden or similar
A phone number you can receive texts on	To test that texts arrive	yours
About EUR 40	First month of hosting and EUR 20 of text-message credit	card

Keep one document (in the password manager) called **Greecon platform**. Every key you create below goes in it the moment you create it. If you lose one, nothing is broken: you make a new one and paste it in again.

Step 1 Three addresses, decided

These are decided so that you do not have to think about them. The second one matters most.

What	Address	Why
The web app (customers, staff, the project manager)	`https://app.greecon.earth`	What people type and bookmark
The platform's own address, which every box at every site reports to	`https://api.greecon.earth`	**A box remembers this address for as long as it lives.** If it ever changes, someone has to visit every box. Choose it once, here, and never change it.
Where alert emails come from	`alerts@greecon.earth`	Replies go to `info@greecon.earth`

Step 2 Railway: the project and its database

1. Sign in at railway.com with GitHub. Add the card.
2. **New Project** → **Deploy from GitHub repo** → choose `greecon-tech/app`. If it asks which branch, choose `main`. Railway will try to build something and probably fail the first time. That is expected: it does not know yet which of the two programs to build. Continue.
3. In the project, click **+ New** → **Database** → **Add PostgreSQL**. Railway creates the database.
   - This platform needs PostgreSQL 14 or newer (Railway's current default is newer than that).
4. **Region:** where Railway offers a choice (project or service **Settings**), choose a **European** region for the services and the database, so that customers' data stays in Europe. A database's region is fixed when it is made, so choose before you add it.

You should see: a project with two boxes, your repository's service and `Postgres`, with Postgres showing as active. Rename the repository's service to **api** (click it, Settings, Service Name). The names matter below.

Step 3 Railway: the API service

Click the service you named `api`.

Settings → **Build** → **Dockerfile Path:** `infra/docker/api.Dockerfile`

Settings → **Deploy** → **Healthcheck Path:** `/health` (this lets Railway see that a new version is alive before it switches to it).

Variables. Add these one at a time (use **Raw Editor** if Railway offers it and paste the block). The first column is the name, exactly as written.

Name	Value	What it does
`DATABASE_URL`	`${{Postgres.DATABASE_URL}}`	Type it exactly like that, with the braces. Railway fills in the database's address.
`JWT_SECRET`	a long random value, at least 32 characters	Signs everyone's session. Make one in your password manager ("generate password", 48 characters) and paste it. **If it is short or missing, the platform refuses to start, on purpose.**
`PORT`	`4000`	The door the platform listens on
`TRUST_PROXY`	`1`	Lets the platform see each caller's real address behind Railway
`PUBLIC_API_URL`	`https://api.greecon.earth`	The address written into every box's install command
`PUBLIC_WEB_URL`	`https://app.greecon.earth`	The link in each alert message
`BOOTSTRAP_ADMIN_EMAIL`	`eridon.manuka@greecon.earth`	Your own login, created on the first start. **Removed in step 6.**
`BOOTSTRAP_ADMIN_PASSWORD`	a password of at least 12 characters, not used anywhere else	Your first password. **Removed in step 6.**
`SALES_NOTIFY_EMAIL`	`info@greecon.earth`	Who is emailed when someone asks for access

Messages are added in steps 7 and 8. Do not set `ENABLE_API_DOCS`, `AUTO_MIGRATE` or the old `EDGE_INGEST_*` variables.

Settings → Networking → Generate Domain gives the API a temporary Railway address (ending `.up.railway.app`). Use it only to check that the API is alive, below. Then **Custom Domain → `api.greecon.earth`**: Railway shows a record to add to your DNS (step 5).

You should see: after a minute or two, in the service's **Deployments → View logs**, a line such as `Database updated: 17 migration(s) applied, 0 recorded as already applied.` (on later deploys, `Database is up to date.`), then `Nest application successfully started`. Also, the first time:

```
Password set for eridon.manuka@greecon.earth from BOOTSTRAP_ADMIN_PASSWORD. Sign in, change it in Settings, then remove both BOOTSTRAP_ADMIN variables.
```

(On a database that never had your account the line says "Administrator … created from BOOTSTRAP_ADMIN_PASSWORD" instead; either means it worked.) Open `https://<the temporary address>/health` in a browser: it must show `"status":"OK"` and `"database":"connected"`.

Warnings in the log are information, not failures. The platform lists what is not set up yet (messages are off, and so on). The log line "The deployment's settings are not safe to start with" is a failure: it names the setting to fix. The usual cause is a `JWT_SECRET` that is too short.

Step 4 Railway: the web service

1. **+ New → GitHub Repo →** `greecon-tech/app` again, same branch. Name it **web**.
2. **Settings → Build → Dockerfile Path:** `infra/docker/web.Dockerfile`
3. **Variables:** one only.

Name	Value
`GREECON_API_URL`	`http://${{api.RAILWAY_PRIVATE_DOMAIN}}:4000`

   (If you did not name the first service `api`, put its name in place of `api`.) This is the private road between the two services. The API does not need to be reachable by the web app through the internet.
4. **Settings → Networking → Generate Domain**, then **Custom Domain →** `app.greecon.earth`.

You should see: the web service deploys and its temporary address shows a sign-in page titled "Greecon Platform".

Step 5 DNS: the two addresses

At wherever `greecon.earth` is managed, add the two records Railway showed you (one for `api`, one for `app`). They are usually `CNAME` records pointing at an address Railway gives. Railway shows a green tick when it sees them, and then issues the padlock (HTTPS) by itself. This can take from a few minutes to an hour.

You should see: `https://api.greecon.earth/health` shows `"status":"OK"` and `"database":"connected"`, and `https://app.greecon.earth/login` shows the sign-in page with a padlock.

If it never turns green: the record was added to the wrong name (it must be `api` and `app`, not `api.greecon.earth.greecon.earth`), or the DNS host puts a proxy in front of it. Switch any proxy or "orange cloud" off for these two names.

Step 6 First sign-in, and removing the starting password

1. Open `https://app.greecon.earth/login`. Click **Greecon team**. Enter `eridon.manuka@greecon.earth` and the starting password. **Continue.**
2. You land on **Today**, Greecon's own start page. With no customer yet it says *No customer site is live yet.* If you see it, the platform works end to end.
3. Open **Settings**, then its **Account** tab, and under **Change your password** use **Change password** to choose your own, long and not used anywhere else. Put it in the password manager.
4. **Remove the two `BOOTSTRAP_ADMIN_` variables** from the API service in Railway. Leaving them is not dangerous (they only fill a gap and never replace a password), but there is no reason to leave a password in a settings page.
5. Give the project manager a login of their own: open **Team**, and under **Greecon team** enter their name and email and press **Add to the team**. A temporary password is shown **once**. Tell it to them by phone or in person, not by email. They sign in with **Greecon team**, and are asked for a password of their own straight away.

Only the owner (you) can add or remove team members. A project manager can see the team and cannot change it.

Step 7 Email (Resend)

Email is how nearly every message reaches a person: Act now, Check today, the morning message, and the note that tells you someone asked for access.

1. Sign up at resend.com (free plan: 100 emails a day, 3,000 a month, one domain: enough for the first five sites).
2. **Domains** → **Add Domain** → `greecon.earth` (or a subdomain such as `alerts.greecon.earth`, which keeps this mail apart from your own). Resend shows several DNS records (they prove the mail really comes from you, which is what keeps it out of spam). Add every one of them where `greecon.earth` is managed, then click **Verify**. It can take up to an hour.
3. **API Keys** → **Create API Key** → "Sending access", for that domain. Copy it (it begins `re_`) into the password manager.
4. In Railway, on the **api** service, add:

Name	Value
`RESEND_API_KEY`	the key
`ALERT_FROM_EMAIL`	`alerts@greecon.earth` (or `alerts@alerts.greecon.earth` if you used the subdomain)
`ALERT_FROM_NAME`	`Greecon`
`ALERT_REPLY_TO`	`info@greecon.earth`

   Railway redeploys the service.
5. **Test it:** sign in, open **Settings** and its **Who is told** tab, add yourself with your email, **Save the list**, then **Send a test message**. It should say "Accepted for delivery" and the email should arrive within a minute. Look in spam too. If it is in spam, mark it "not spam" and ask the Resend support page about your domain's setup (a missing record is the usual reason).

When the free plan is not enough: a hundred emails a day is reached at about fifteen sites. The paid plan is about USD 20 a month for 50,000. Nothing else changes.

Step 8 Text messages (Twilio)

Text messages are for **Act now** only: a tank nearly empty, a pump running with no water moving (where there is a flow meter). The first person is texted ten minutes after the email if nobody has acknowledged the alert (by pressing **Acknowledge** in Greecon: replying to the text does nothing), and the next person is emailed and texted thirty minutes after the alert began, and so on down the list.

1. Sign up at twilio.com. Verify your own phone. Add about EUR 20 of credit (a text to Albania costs a few cents; check Twilio's price page for Albania).

2. You need something to send *from*. Two ways; choose one:
   - **A name:** `Greecon`. Twilio lists Albania as a country where a sender name can be used without registering first, and a name is what Albanian recipients are used to seeing. A text sent from a name cannot be replied to. Some Albanian operators have been reported to replace a name with a generic one.
   - **A number:** buy a Twilio number that can send to Albania. Replies can be received, and the sender is the same everywhere. **Test before you decide**, with phones on all four networks (One, Vodafone, Telekom and Plus, whichever your customers use).
3. In Railway, on the **api** service, add:

Name	Value
`TWILIO_ACCOUNT_SID`	from the Twilio dashboard (begins `AC`)
`TWILIO_AUTH_TOKEN`	from the Twilio dashboard. Treat it like a password.
`TWILIO_FROM`	`Greecon`, or the number in international form, such as `+1...`

4. **Test it:** on **Who is told** (in **Settings**), add yourself with your mobile number *with the country code* (`+355 69 ...`), save, and **Send a test message**. A text should arrive within a minute. Ask three people on other networks to try it too.

If the text does not arrive: Twilio's console shows each message and why it was refused (**Monitor → Logs → Messaging**). The commonest reasons are a number not written with its country code and a sender name a network does not accept. **What was sent**, under **Who is told**, shows what Greecon asked Twilio to do.

You can launch without texts. Then only email carries Act now. Say so to customers, and do it quickly: texts are the reason a manager wakes up at night. The setup checklist will not let a site be commissioned until at least one of the two ways works.

Step 9 Greecon's own morning check

Greecon must hear that a customer's box has gone quiet before the customer does.

1. Sign in as yourself (**Greecon team**), open **Settings**, then **Who is told**. This is Greecon's own list.
2. Add yourself and the project manager, each with an email address. **Save the list.**
3. Leave the morning message on. It is sent at 07:30, Albania time (change it under the list if you prefer).

Each morning you get one email: "Greecon, Mon 19 Oct: Stable. All 3 sites are reporting and nothing needs doing." or "Attention needed. 1 of 3 sites need a look." with the reasons in the line for each site. The same list is on **Today**, under **Needs a look**, worst first; the sites that are fine are folded away below it, and each live site shows how many days it has been live (*Day N*). The runbook (the Complete Deployment Guide, Part 7 (PDF 00)) says what to do.

Step 10 The demo

Two demos, for two uses.

The one that is always there, with no sign-in: `https://greecon-tech.github.io/app/`. A frozen picture of the product with simulated data, one page per kind of user. To switch it on once: on GitHub, in `greecon-tech/app`, **Settings** → **Pages** → **Build and deployment** → **Source: GitHub Actions**. After the next change to `main`, it publishes by itself. Use this for the link in an email.

The live one, for a meeting: sign in at `https://app.greecon.earth/login` with **Client login**, email `demo@greecon.earth`, password `demo123`. It shows three simulated sites, a farm, a water site and a solar-plus-battery site (there is no hotel in it), that move as the day goes (the solar picture follows the sun). It holds nothing real, it can only look and try, and it cannot send anyone a message. This login is an **operator**: it has no **Who is told**, and its **Reports** shows **Trends** only, with no **Downloads**, so those two are shown on your own rehearsal client instead (the Sales kit (document 04, not part of these guides), section 4). The password is public on purpose; do not give it to a customer as their own login.

Step 11 Watching it

Do this	How	Why
An outside check that the platform is up	Make a free account at uptimerobot.com or betterstack.com. Add a monitor for `https://api.greecon.earth/health` every 5 minutes that looks for the word `connected` (a "keyword" monitor), alerting your email and phone: it then also notices a database that has gone. Add one for `https://app.greecon.earth/login`.	If the whole platform is down, nothing inside it can tell you. This is the only thing that can.
Backups	In Railway, open the Postgres service's **Backups** tab and turn on a daily one (what your plan allows: Railway's scheduled backups may need the paid Pro plan, which is worth it once a customer is live). In addition, once a week, take a copy off Railway: the Complete Deployment Guide, Part 7 (PDF 00), "Backups and restoring".	A backup that only lives next to the database does not survive losing it.
Calendar reminders	Today plus 6 months: re-check the legal list (document 22). 30 September 2027: renew `security.txt` (it carries an expiry date).	Things that expire quietly
Look at the cost	After a week, Railway's **Usage** page. Expect about EUR 20 to 40 a month.	No surprises

Step 12 The rehearsal, then the checklist

Before the first real customer, the project manager does the whole job once on a desk, with a laptop standing in for the meters: the Project Manager Field Manual (PDF 02), “Rehearsal”. It takes an afternoon, and it finds the mistakes in the room instead of at a hotel.

If something goes wrong

What you see	What it means	What to do
The API’s log says “The deployment’s settings are not safe to start with”	A setting is missing or weak	Read the line above it: it names the setting. Usually `JWT_SECRET` shorter than 32 characters.
The web app shows an error on every page	The web app cannot reach the API	Check `GREECON_API_URL` on the **web** service, and that the API service is running
Sign-in says “Invalid email or password”	Wrong password, or the account was turned off	Check the password manager. If it was lost: set `BOOTSTRAP_ADMIN_EMAIL` and `BOOTSTRAP_ADMIN_PASSWORD` again **only if the account has no password**; otherwise ask another owner to reset it, or see the Complete Deployment Guide, Part 7 (PDF 00) (its last point).
Everyone is suddenly sent to the sign-in page	`JWT_SECRET` was changed, which signs everyone out	Nothing is broken. They sign in again.
The install command on a site’s setup page starts with `http://` and a private name instead of `https://api.greecon.earth` (the API’s log also warns of it at start-up)	`PUBLIC_API_URL` is missing	Set it (step 3), redeploy, then make a new code
Emails say “not sent: email is not set up”	`RESEND_API_KEY` is missing or the sender is wrong	Step 7. The API’s log names the reason.
Railway shows the service crashed and restarting	Something in the log	Open the logs and read the last red line. the Complete Deployment Guide, Part 7 (PDF 00), “The platform is down”.

Changing things later

- **Rotate the signing secret** (if you think it leaked, or when a team member leaves who knew it): change `JWT_SECRET` on the API. Everyone is signed out and signs in again. Nothing else changes.
- **Go back to the previous version:** Railway, the service, **Deployments**, the earlier one, **Redeploy**. The database is not rolled back; new versions only add to it, so the old version still works with it.

- **Hold every box on the software it has** (while a problem is investigated): set `EDGE_AUTO_UPDATE` to `false` on the API. Boxes then stop taking new versions until it is removed.
- **Turn the live demo's simulated feed off:** set `DEMO_FEED` to `false`. Only the demo account changes.
- **Move to Google Cloud:** when this outgrows Railway, see Deployment on Google Cloud (document 10). The services are ordinary containers; nothing here ties them to Railway.

What not to do

- Do not give a customer the demo login, and do not share your own login. Everyone has their own.
- Do not change `api.greecon.earth` once a box exists.
- Do not put a key or a password in an email, a chat or a document that is not the password manager.
- Do not hand out hardware or install anything at a customer before the checklist above is ticked.

TEAR OFF AND KEEP

Go-live checklist

Tick each one yourself; none needs a technician.

- [ ] `https://api.greecon.earth/health` shows `"status":"OK"` and `"database":"connected"`, and `https://app.greecon.earth/login` shows a padlock
- [ ] I signed in with **Greecon team** and saw **Today**
- [ ] The two `BOOTSTRAP_ADMIN_` variables are gone, and my password is in the password manager
- [ ] The project manager signed in with their own login and chose their own password
- [ ] A test email reached my inbox (not spam), and says it is from `alerts@greecon.earth`
- [ ] A test text reached my phone and the phones of three people on other networks (or I have decided to launch with email only, and said so)
- [ ] **Who is told** in Greecon's own account has me and the project manager; tomorrow morning I get the morning check
- [ ] `https://greecon-tech.github.io/app/` opens the demo
- [ ] The outside check is watching both addresses and has my phone number
- [ ] A backup exists (the Backups tab shows one, or I took one by hand and put it in the shared drive)
- [ ] `https://app.greecon.earth/privacy` opens without signing in, and says what Greecon keeps (the lawyer reads it before the first customer signs)
- [ ] I sent myself a "Request access" from the sign-in page and it arrived at `info@greecon.earth` and appeared under **Requests** (and in **Requests waiting** on **Today**)
- [ ] The rehearsal was done, and its notes are fixed or written down

Signed: _____ Date: _____